

Greedy Algorithms

Complete Guide

1. What is it?

Greedy is a technique where at **each step you make the locally optimal choice** — the choice that looks best *right now* — without reconsidering it later, and trust that the sequence of locally optimal choices leads to a globally optimal solution.

Unlike DP, greedy never backtracks or explores alternatives — one pass, one decision per step, done.

2. Why is it used?

- Much simpler and faster than DP or brute force when it's provably correct (often $O(n \log n)$ vs $O(n^2)$ or exponential)
- No extra memory for memoization tables
- Easy to reason about and implement once you've identified the right “greedy criterion” (what to sort by / what to pick first)

3. Where is it used?

- Interval scheduling (max non-overlapping intervals, min intervals to remove)
- Activity selection / meeting attendance
- Jump game (reachability, min jumps)
- Gas station / circular tour feasibility
- Huffman encoding, minimum spanning tree (Kruskal's, Prim's)
- Task scheduling with cooldowns
- Assigning cookies / candies to children (two-pointer + greedy combo)
- Stock buy/sell (single pass, track running min/max)
- Coin change (only when denominations are canonical, e.g. US coins — NOT in general)
- Huffman-style priority queue problems (merge stones, connect ropes)

4. How to identify a Greedy problem (pattern triggers)

Signal in the problem statement	What it suggests
"maximum/minimum number of ..." with simple constraints	Possible greedy — check if local choice is provably safe
"non-overlapping intervals"	Sort by end time, greedily pick earliest-ending
"minimum number of steps/jumps/removals"	Greedy reachability or interval-removal
"assign X to Y to satisfy as many as possible"	Sort both sides, two-pointer greedy matching
"at each step choose the best/cheapest/largest available"	Explicit greedy phrasing
Problem has an exchange argument feel — swapping two choices never makes it worse	Strong greedy signal
Optimal substructure but NO overlapping subproblems (each choice fully determines the rest, no need to "remember" alternatives)	Greedy over DP

Paraphrase test: If you can rephrase the problem as "if I always pick the best option available right now, will I never regret it later?" and you can convince yourself yes (via an exchange argument or contradiction), it's greedy.

5. Can you tell from constraints alone?

Yes, partially:

- If expected complexity is $O(n \log n)$ and DP would need $O(n^2)$ or worse, that's a hint towards greedy + sort.
- Greedy problems rarely need extra memory beyond $O(1)$ or $O(n)$ for sorting — if the intended solution needs a DP table, it's usually not pure greedy.

Caution: constraints alone can mislead you — many problems *look* greedy but are DP in disguise (e.g. general coin change, 0/1 knapsack). Constraints only tell you complexity expectations, not correctness of the greedy approach — you must verify the greedy choice is provably safe (exchange argument or matroid structure), not just assume it works because it's fast.

6. Intuition / Rationale

Greedy works when the problem has two properties: 1. **Greedy choice property** — a locally optimal choice can always be extended to a globally optimal solution (proved via an exchange argument: assume an optimal solution doesn't make the greedy choice, show you can swap it in without making things worse). 2. **Optimal substructure** — after making the greedy choice, the remaining subproblem is the same type of problem, just smaller.

The exchange argument is the core proof technique: take any optimal solution, show that replacing its first decision with the greedy decision doesn't hurt (or only helps), and repeat inductively. If you can't construct this argument for the problem — check DP instead.

Classic pitfall: 0/1 knapsack looks greedy (pick items with best value/weight ratio) but isn't — a locally best item can block a better combination later, because items can't be split. Fractional knapsack (where cutting off arbitrary weight is allowed) IS greedy, because you never lose potential from a partial choice.

7. Time & Space Complexity (generic)

- **Time:** $O(n \log n)$ if sorting is required first, $O(n)$ if no sort needed (e.g. single-pass running-min/max problems)
- **Space:** $O(1)$ extra beyond input/output, or $O(n)$ if a heap/priority queue is used

8. Types of Greedy (with Java templates)

Type A — Interval Scheduling (sort by end time)

Used for: max non-overlapping intervals, min removals to make non-overlapping, activity selection.

```
public int eraseOverlapIntervals(int[][] intervals) {
    if (intervals.length == 0) return 0;
    Arrays.sort(intervals, (a, b) -> a[1] - b[1]); // sort by END time

    int count = 0;
    int lastEnd = intervals[0][1];
    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] < lastEnd) {
            count++; // overlap -> remove this interval (greedy: keep the one ending earlier)
        } else {
            lastEnd = intervals[i][1];
        }
    }
    return count;
}
```

Type B — Single-Pass Running Min/Max

Used for: best time to buy/sell stock, gas station feasibility.

```
public int maxProfit(int[] prices) {  
    int minPriceSoFar = Integer.MAX_VALUE;  
    int maxProfit = 0;  
    for (int price : prices) {  
        minPriceSoFar = Math.min(minPriceSoFar, price); // greedy: track cheapest buy point  
        maxProfit = Math.max(maxProfit, price - minPriceSoFar);  
    }  
    return maxProfit;  
}
```

Type C — Reachability / Jump Game (extend the frontier greedily)

Used for: jump game, minimum jumps to reach end.

```
public boolean canJump(int[] nums) {
    int farthestReachable = 0;
    for (int i = 0; i < nums.length; i++) {
        if (i > farthestReachable) return false; // stuck, can't even reach index i
        farthestReachable = Math.max(farthestReachable, i + nums[i]); // greedy: extend as far as possible
    }
    return true;
}
```

Type D — Two-Sorted-Array Matching (greedy pairing)

Used for: assign cookies, boats to save people (pair smallest with largest under a limit).

```
public int findContentChildren(int[] g, int[] s) { // g = children's greed factors, s = cookie sizes
    Arrays.sort(g);
    Arrays.sort(s);
    int child = 0, cookie = 0;
    while (child < g.length && cookie < s.length) {
        if (s[cookie] >= g[child]) {
            child++; // this cookie satisfies this child, greedily move to next child
        }
        cookie++; // this cookie is used up either way
    }
    return child;
}
```

Type E — Heap-Based Greedy (always combine the two smallest/best)

Used for: merge stones/ropes with minimum cost, Huffman-style problems.

```
public int connectRopes(int[] ropes) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    for (int r : ropes) minHeap.offer(r);

    int totalCost = 0;
    while (minHeap.size() > 1) {
        int first = minHeap.poll();
        int second = minHeap.poll();
        int cost = first + second;
        totalCost += cost;
        minHeap.offer(cost); // greedy: always merge the two cheapest ropes first
    }
    return totalCost;
}
```

9. Comparison of the patterns

Type	Core greedy criterion	Typical trigger words	Complexity
A. Interval scheduling	sort by end, keep earliest finisher	"non-overlapping", "max activities", "min removals"	$O(n \log n)$
B. Running min/max	track best-so-far in one pass	"buy/sell", "max profit", "single pass"	$O(n)$
C. Reachability frontier	extend farthest reachable index	"can you reach", "min jumps"	$O(n)$
D. Two-sorted-array matching	pair smallest feasible match	"assign", "satisfy as many as possible"	$O(n \log n)$
E. Heap-based merge	always combine two cheapest/smallest	"minimum cost to combine/merge", "Huffman"	$O(n \log n)$

Key distinguishing question: Am I choosing between overlapping ranges ($\rightarrow$ A), tracking a best value seen so far ($\rightarrow$ B), extending how far I can go ($\rightarrow$ C), matching two separate sorted lists ($\rightarrow$ D), or repeatedly combining the two cheapest items ($\rightarrow$ E)?

10. Quick decision checklist (use this in an interview)

- 1 Can I sort the input by some criterion (end time, ratio, value) such that picking greedily in that order is safe? $\rightarrow$ **Type A/D**
- 2 Am I just tracking a running best value while scanning once? $\rightarrow$ **Type B**
- 3 Is the question "how far can I get" or "is X reachable"? $\rightarrow$ **Type C**
- 4 Do I need to repeatedly combine the two best/cheapest available options? $\rightarrow$ **Type E** (heap)
- 5 **Can I construct an exchange argument** — "if the optimal solution didn't make this choice, I could swap it in without making things worse"? If yes $\rightarrow$ greedy is provably correct. If no, and especially if items are indivisible/discrete with interacting constraints $\rightarrow$ suspect **DP** instead.

11. Comparison with Two Pointers and Sweep Line

	Two Pointers	Sweep Line	Greedy
Core idea	Two indices moving through one sorted array	Process sorted events, track running state	Make locally optimal choice at each step, never revisit
Correctness proof	Monotonicity of the invariant (one move only ever helps)	Same monotonicity, applied to event order	Exchange argument / optimal substructure
Typical output	Pair/triplet, boolean, optimal value	Count of overlaps, min resources, shape	Optimal count/cost/sequence of picks
Overlap with greedy	Type D “two-sorted-array matching” above is literally a two-pointer template used greedily	Type A interval sweep and Type A greedy interval scheduling are nearly the same code, different framing	—

In practice, greedy and two pointers/sweep line often **coexist in the same solution** — sorting the input is a greedy setup step, and the pointer/sweep mechanics are how you execute the greedy choices efficiently. If you're identifying a problem as “greedy,” it's worth also asking whether the actual mechanics resemble a two-pointer or sweep-line implementation.